

Índice

1. Descripción del Proyecto	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Marco Teórico	2
3.1. Metaheurísticas	2
3.2. Algoritmo ACO	2
3.3. Feromonas	3
3.4. Movimiento probabilístico	3
3.5. GPU y CUDA	3
4. Metodología	3
4.1. Configuración del entorno	3
4.2. Construcción del mapa	3
4.3. Inicialización de hormigas	3
4.4. Movimiento de las hormigas	4
4.5. Depósito de feromonas	4
4.6. Evaporación de feromonas	4
4.7. Implementación en GPU	4
5. Resultados	4
5.1. Evolución del algoritmo	4
6. Análisis de resultados	5
7. Conclusiones	5
8. Repositorio o Evidencia de Ejecución	5

1. Descripción del Proyecto

El presente proyecto tiene como objetivo implementar el algoritmo Ant Colony Optimization (ACO), una metaheurística bio-inspirada basada en el comportamiento de las colonias de hormigas, para encontrar una ruta óptima entre un punto de origen y un punto de destino dentro de una matriz con obstáculos.

El sistema simula cómo un conjunto de hormigas artificiales explora un espacio, toma decisiones de movimiento de manera probabilística y deposita feromonas sobre los caminos recorridos. A medida que avanzan las iteraciones, los caminos más eficientes reciben mayor cantidad de feromonas, mientras que los caminos menos favorables pierden importancia debido al proceso de evaporación.

La implementación se desarrolló en Python utilizando matrices para representar el espacio del experimento, obstáculos, feromonas y rutas. Además, se empleó **Numba CUDA** para acelerar el procesamiento mediante GPU, permitiendo simular múltiples hormigas en paralelo.

2. Objetivos

2.1. Objetivo general

Implementar un sistema basado en el algoritmo ACO para encontrar una ruta de menor costo entre una colonia de hormigas y una fuente de comida, utilizando aceleración por GPU.

2.2. Objetivos específicos

- Representar el espacio del experimento mediante una matriz bidimensional.
- Definir obstáculos, punto de origen y punto de destino.
- Simular el movimiento probabilístico de múltiples hormigas.
- Implementar el depósito y evaporación de feromonas.
- Utilizar CUDA para ejecutar el algoritmo en GPU.
- Visualizar la ruta encontrada y la evolución de las feromonas.

3. Marco Teórico

3.1. Metaheurísticas

Una metaheurística es un método de búsqueda utilizado para encontrar soluciones aproximadas a problemas complejos de optimización. Aunque no garantiza encontrar siempre la solución óptima global, permite obtener soluciones de buena calidad en tiempos razonables.

3.2. Algoritmo ACO

El algoritmo *Ant Colony Optimization* se inspira en el comportamiento de las hormigas reales. Estas hormigas no poseen un mapa del entorno, pero logran encontrar rutas cortas hacia la comida gracias al uso de feromonas.

Cuando una hormiga encuentra una ruta hacia la fuente de comida, deposita feromonas sobre el camino recorrido. Las demás hormigas tienden a seguir los caminos con mayor cantidad de feromonas, haciendo que las rutas más cortas se refuercen con el tiempo.

3.3. Feromonas

Las feromonas son valores numéricos almacenados en una matriz. Estos valores influyen en la decisión de movimiento de las hormigas. Una celda con mayor nivel de feromonas tiene mayor probabilidad de ser elegida.

El proceso de evaporación reduce gradualmente las feromonas:

$$\tau_{i,j} = (1 - \rho)\tau_{i,j}$$

donde $\tau_{i,j}$ representa el nivel de feromona y ρ es el factor de evaporación.

3.4. Movimiento probabilístico

Cada hormiga selecciona su siguiente movimiento considerando la feromona y la heurística hacia el destino. La probabilidad de moverse desde una celda i hacia una celda j está dada por:

$$P_{i,j} = \frac{(\tau_{i,j})^\alpha (\eta_{i,j})^\beta}{\sum (\tau_{i,u})^\alpha (\eta_{i,u})^\beta}$$

donde τ es la feromona, η es la heurística, α representa la importancia de la feromona y β representa la importancia de la heurística.

3.5. GPU y CUDA

Las GPU permiten ejecutar muchas operaciones en paralelo. En este proyecto, cada hormiga puede ser procesada por un hilo diferente, lo cual permite acelerar el cálculo de rutas, movimientos y actualización de feromonas.

CUDA permite escribir funciones llamadas *kernels*, las cuales se ejecutan directamente en la GPU.

4. Metodología

4.1. Configuración del entorno

El proyecto fue desarrollado en Google Colab utilizando Python. Se emplearon las librerías NumPy, Matplotlib y Numba CUDA.

4.2. Construcción del mapa

Se creó una matriz de tamaño $Y \times X$, donde cada celda representa una posición del espacio. Las celdas con valor 0 representan caminos libres, mientras que las celdas con valor 1 representan obstáculos o paredes.

También se definieron dos puntos principales:

- **Colonia:** punto inicial de las hormigas.
- **Fuente de comida:** punto objetivo al que deben llegar.

4.3. Inicialización de hormigas

Se definió una cantidad N de hormigas. Todas inician desde la colonia y comienzan a explorar el espacio en busca de la fuente de comida.

4.4. Movimiento de las hormigas

Cada hormiga evalúa las celdas vecinas disponibles y selecciona su siguiente movimiento con base en una función probabilística. Esta decisión depende del nivel de feromonas y de la distancia hacia el destino.

Además, se evita que las hormigas atraviesen obstáculos y se reducen los ciclos revisando posiciones ya visitadas.

4.5. Depósito de feromonas

Cuando una hormiga logra llegar a la fuente de comida, deposita feromonas sobre el camino recorrido. La cantidad depositada es inversamente proporcional a la longitud del camino:

$$\Delta\tau = \frac{Q}{L}$$

donde Q es una constante y L es la longitud del camino encontrado.

4.6. Evaporación de feromonas

Al finalizar cada iteración, se reduce el nivel de feromonas en toda la matriz. Esto permite que los caminos poco eficientes desaparezcan progresivamente y que nuevas rutas puedan ser exploradas.

4.7. Implementación en GPU

Se implementaron kernels CUDA para ejecutar en paralelo las principales operaciones del algoritmo:

- Movimiento de las hormigas.
- Depósito de feromonas.
- Evaporación de feromonas.

Cada hormiga es procesada por un hilo de la GPU, lo cual permite simular muchas hormigas simultáneamente.

5. Resultados

El algoritmo logró encontrar una ruta desde la colonia hasta la fuente de comida evitando los obstáculos del mapa.

Durante las iteraciones, las rutas más cortas comenzaron a acumular mayor cantidad de feromonas. Esto permitió que el sistema reforzara progresivamente los caminos más eficientes.

También se generaron visualizaciones del mapa, la matriz de feromonas y la mejor ruta encontrada por el algoritmo.

5.1. Evolución del algoritmo

Al inicio, las hormigas exploran el espacio de forma más aleatoria. Sin embargo, a medida que aumenta el número de iteraciones, las feromonas guían el comportamiento colectivo hacia mejores soluciones.

6. Análisis de resultados

El comportamiento observado demuestra el principio de estigmergia, donde las hormigas no se comunican directamente entre sí, sino mediante modificaciones en el entorno. En este caso, dichas modificaciones corresponden a los niveles de feromonas almacenados en la matriz.

El uso de la heurística permitió orientar a las hormigas hacia la fuente de comida, mientras que las feromonas reforzaron las rutas exitosas. La combinación de ambos factores permitió que el algoritmo encontrara caminos válidos y mejorara progresivamente la solución.

La evaporación fue un elemento importante, ya que evitó que el algoritmo se quedara únicamente con rutas iniciales que no necesariamente eran las mejores. Al reducir las feromonas con el tiempo, se permitió explorar nuevas alternativas.

Desde el punto de vista computacional, la GPU resulta útil porque el comportamiento de cada hormiga puede calcularse de manera independiente. Esto permite aprovechar el paralelismo, especialmente cuando se trabaja con una cantidad grande de hormigas o iteraciones.

7. Conclusiones

- Se implementó correctamente el algoritmo ACO para búsqueda de rutas.
- Se representó el entorno mediante una matriz con obstáculos, origen y destino.
- Se simuló el comportamiento de una colonia de hormigas usando feromonas.
- Se aplicó evaporación para disminuir la influencia de caminos poco eficientes.
- Se utilizó Numba CUDA para ejecutar operaciones en GPU.
- Se logró visualizar la ruta final y la evolución del algoritmo.
- El proyecto permitió comprender cómo una metaheurística bio-inspirada puede resolver problemas de optimización.

8. Repositorio o Evidencia de Ejecución

<https://colab.research.google.com/drive/13TQUqLSA7gfpLna7uJpRLkRjMhJvA6Lq?usp=sharing>